

Lecture 4: Upper Confidence Bound for Bandits

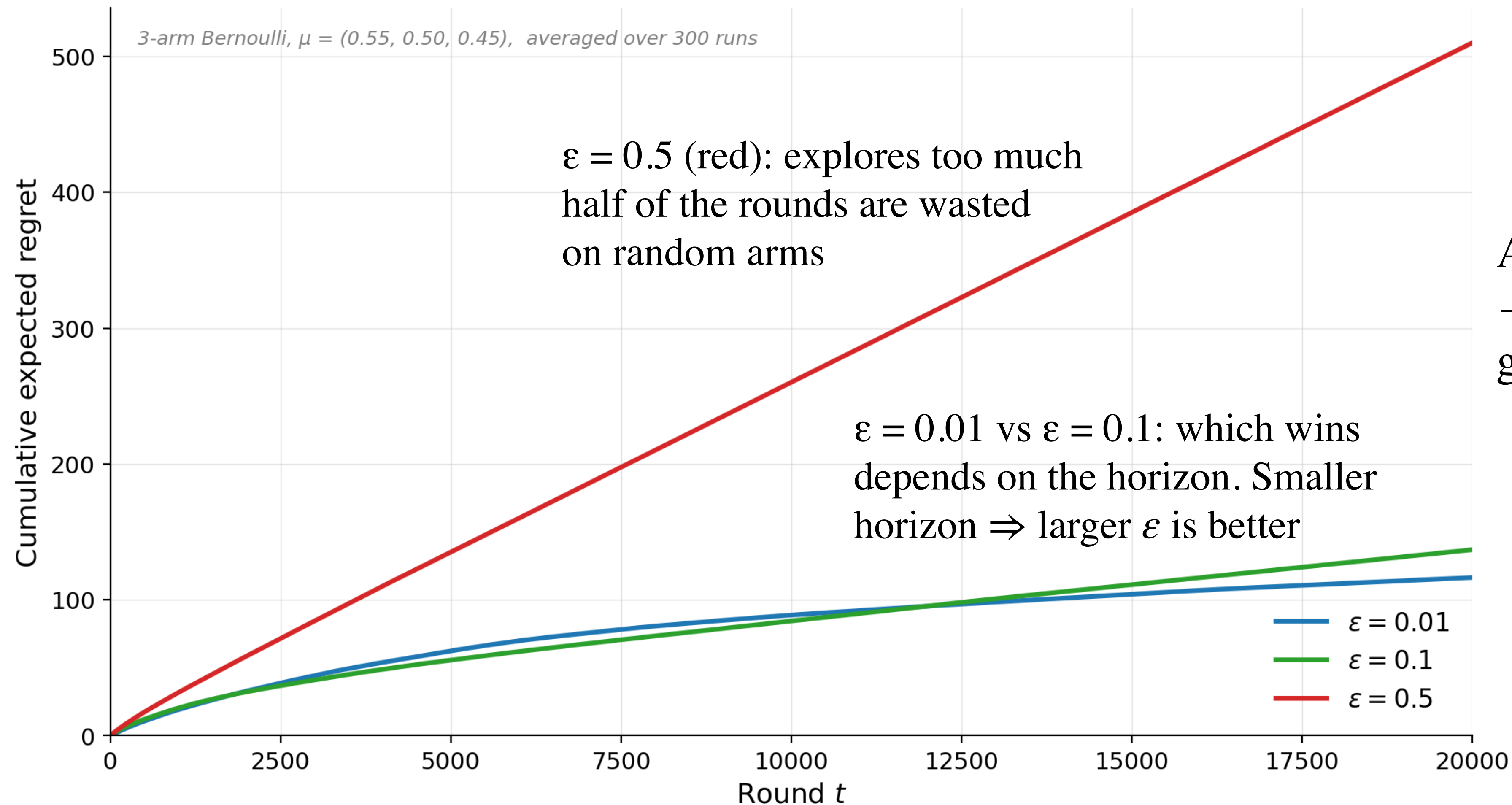
ID 6002W: Online and Reinforcement Learning

Web-enabled M.Tech. program, IIT Madras

**We Already Saw ε -Greedy and
Explore-Then-Commit Algorithms.
Why Do We Need Another Algorithm?**

ϵ -Greedy Algorithm

Behaviour and Limitations



$\epsilon = 0.5$ (red): explores too much
half of the rounds are wasted
on random arms

$\epsilon = 0.01$ vs $\epsilon = 0.1$: which wins
depends on the horizon. Smaller
horizon $\Rightarrow$ larger ϵ is better

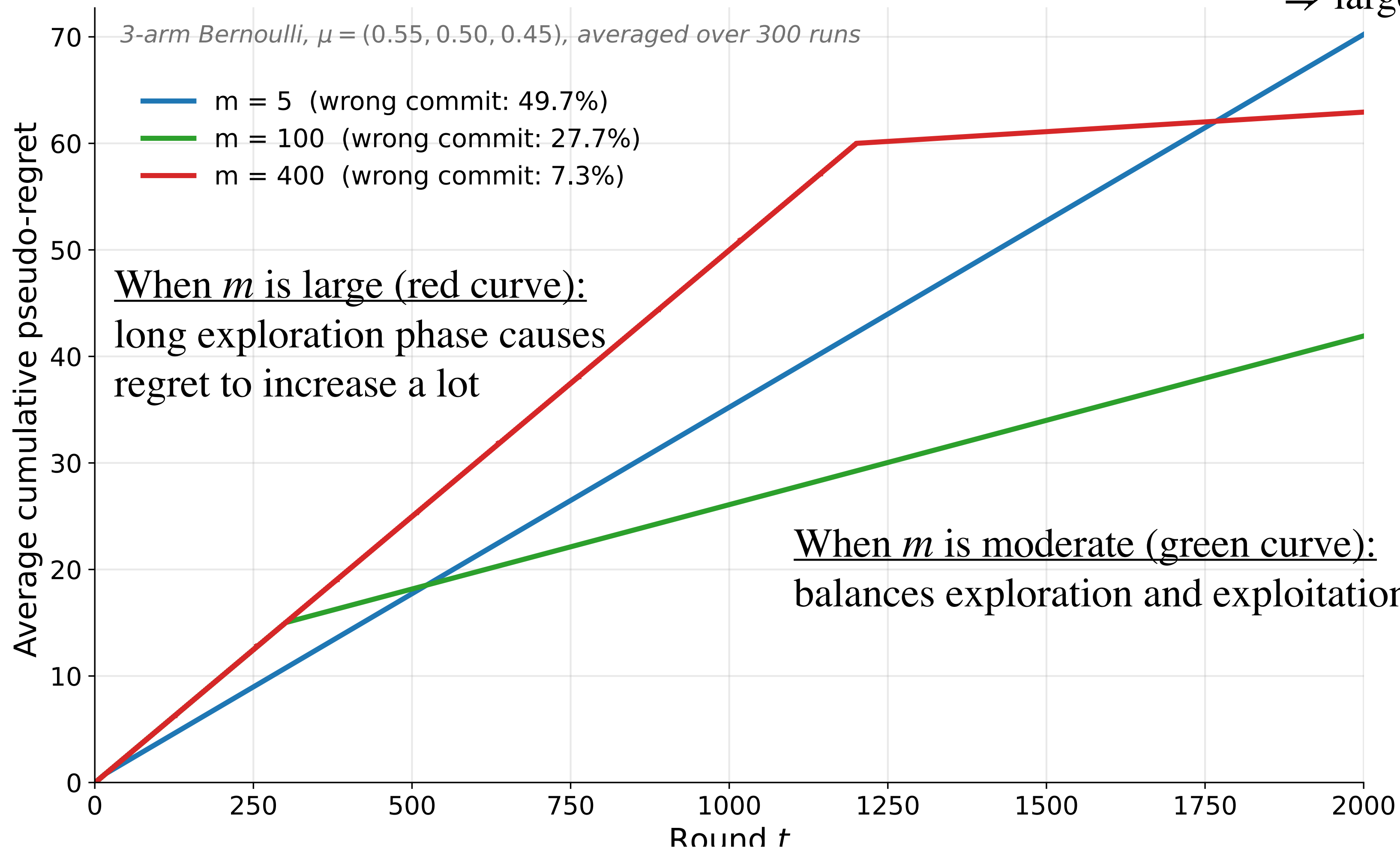
All three curves keep growing
— any fixed ϵ gives linearly
growing regret

No fixed ϵ is universally best

For fixed ϵ , even after we know the best arm, we still explore forever. This leads to linear regret

Explore-Then-Commit

Behaviour and Limitations



When m is small (blue curve):
many runs commit to wrong arm
 $\Rightarrow$ large, linearly growing regret

When m is large (red curve):
long exploration phase causes
regret to increase a lot

When m is moderate (green curve):
balances exploration and exploitation

All three curves keep growing
— any fixed m gives linearly
growing regret

No fixed m is universally best

For any fixed m , there is a chance of permanently committing to the wrong arm. This leads to linear regret

**The Upper Confidence Bound (UCB) Algorithm,
coming up next, smoothly transitions from
exploration to exploitation**

From ETC to UCB

- **ETC:** boundary between exploration and exploitation Km set in advance
- **UCB:** chooses the action to take *based on the data* gathered so far

To explore or to exploit? That is the question

From ETC to UCB

- **ETC:** boundary between exploration and exploitation Km set in advance
- **UCB:** chooses the action to take *based on the data* gathered so far

To explore or to exploit? That is the question

Exploitation: pick the arm with the best $A_t = \arg \max_{i \in \{1,2,\dots,K\}} \hat{\mu}_i$

Explore: pick some other arm

If estimates $(\hat{\mu})$ were reliable, exploiting is the best strategy to minimise regret

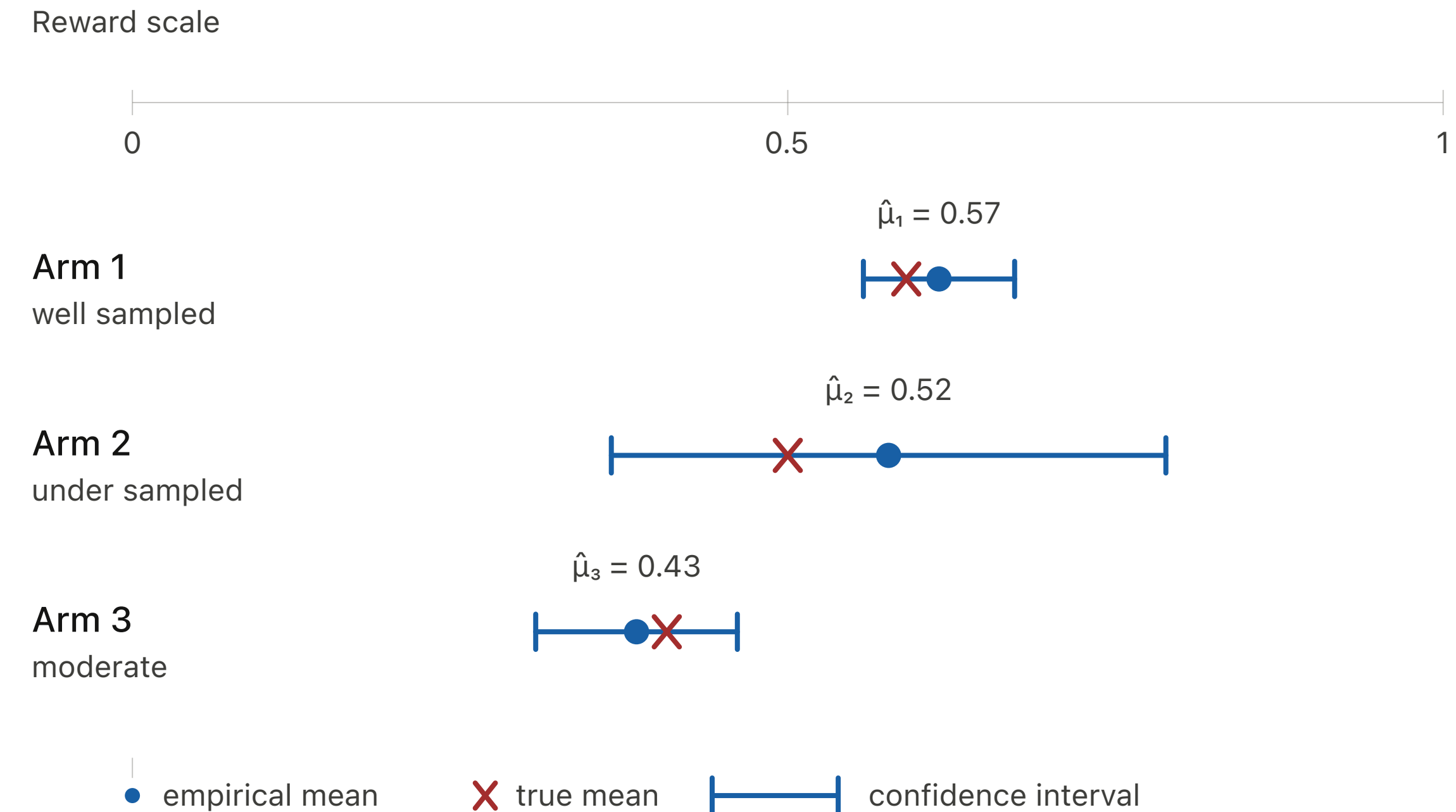
When are estimates not reliable enough?

UCB Algorithm

Using confidence intervals

For each arm i , we can easily maintain

- $\hat{\mu}_i(t)$: estimate of the mean reward μ_i
- $\sqrt{\frac{c \log T}{T_i(t)}}$: width of confidence interval



UCB Algorithm

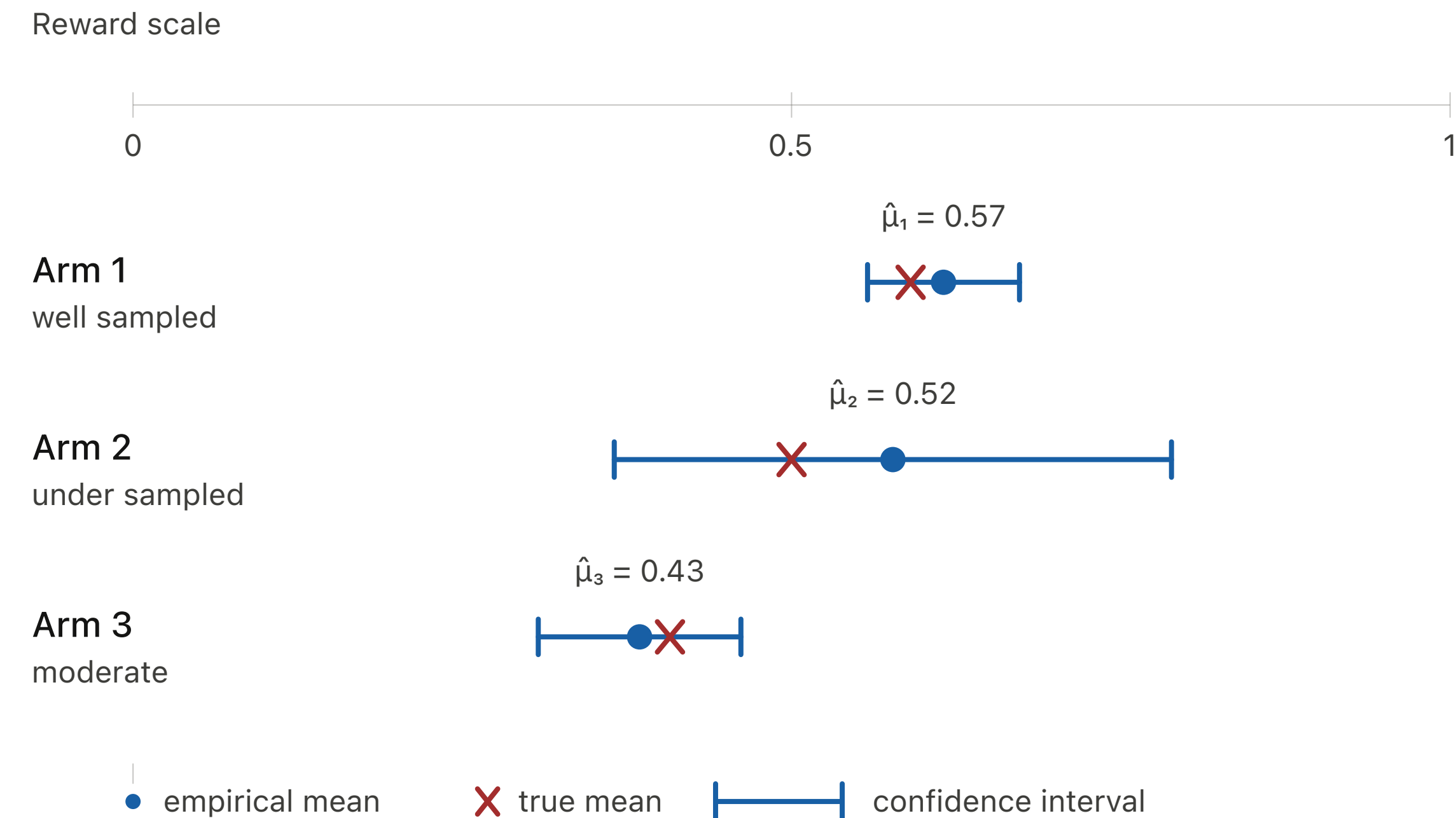
Using confidence intervals

For each arm i , we can easily maintain

- $\hat{\mu}_i(t)$: estimate of the mean reward μ_i
- $\sqrt{\frac{c \log T}{T_i(t)}}$: width of confidence interval

Note:

- Earlier, confidence intervals had $\sqrt{\log(2/\delta)/2n}$. For a fixed horizon T , we take failure probability $\delta \propto 1/T$; constants absorbed in c
- Think of c as a hyper parameter, controlling the amount of exploration

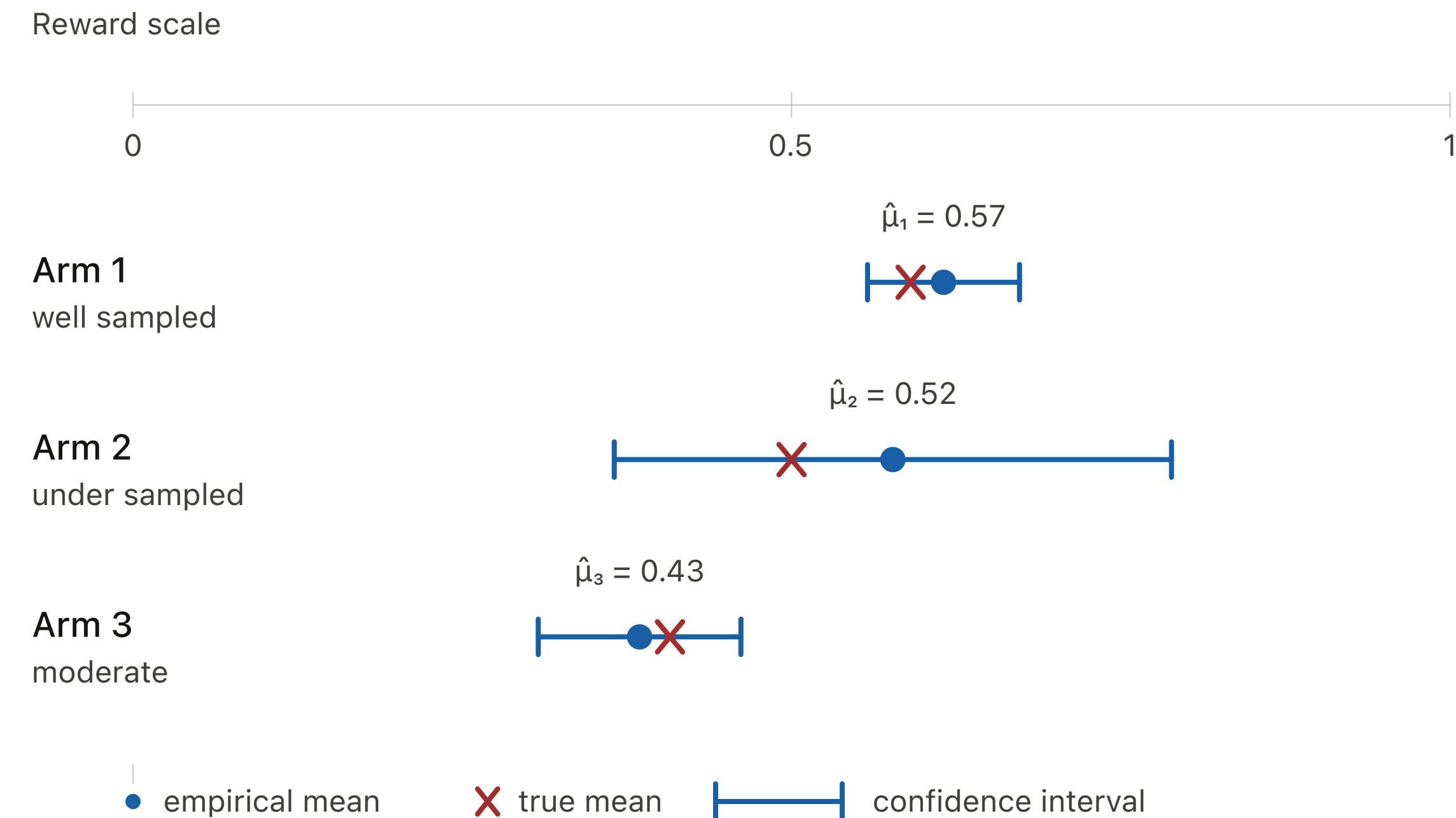


UCB Algorithm

Using confidence intervals

For each arm i , we can easily maintain

- $\hat{\mu}_i(t)$: estimate of the mean reward μ_i
- $\sqrt{\frac{c \log T}{T_i(t)}}$: width of confidence interval



Large confidence intervals imply unreliable estimates

UCB Algorithm

Using confidence intervals

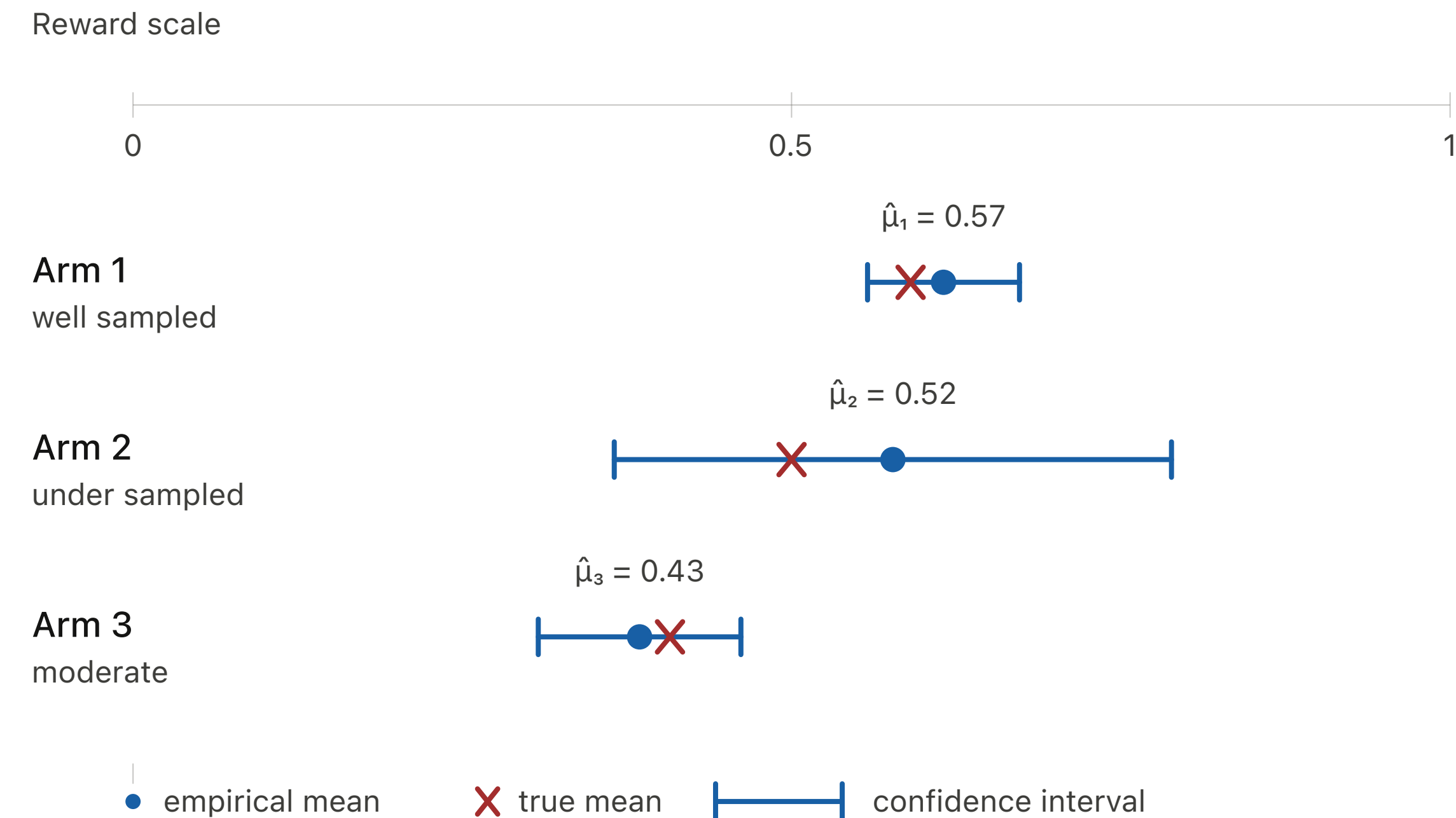
For each arm i , we can easily maintain

- $\hat{\mu}_i(t)$: estimate of the mean reward μ_i
- $\sqrt{\frac{c \log T}{T_i(t)}}$: width of confidence interval

Large confidence intervals imply unreliable estimates

Need to pull those arms more often in order to reduce uncertainty

Pulling all arms equally is suboptimal (ETC). What is a data-driven approach?

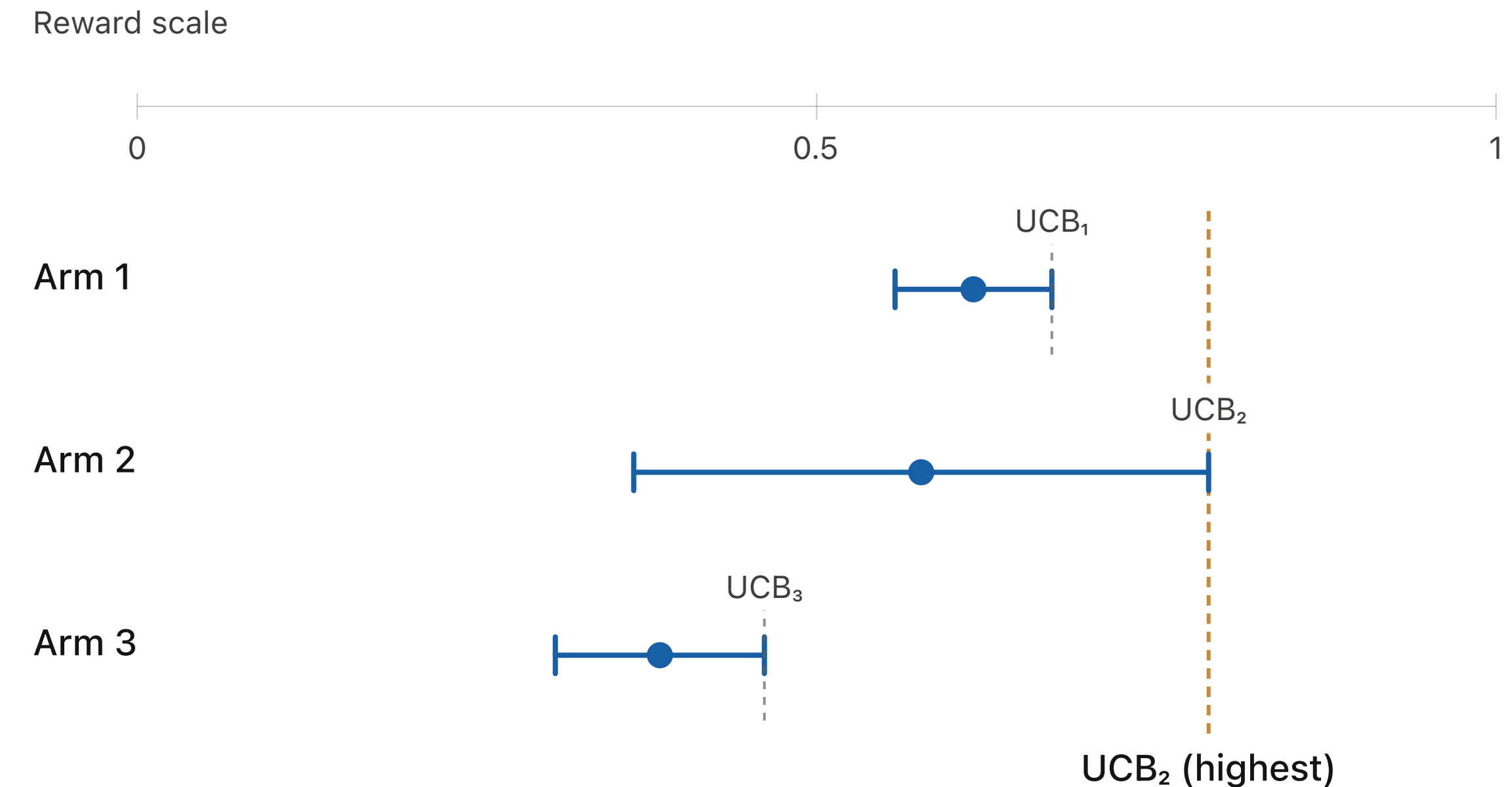


UCB Algorithm

The decision rule

$$UCB_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{c \log T}{T_i(t)}}$$

$$\text{Pull } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t-1)$$



Guiding Principle: Optimism in the Face of Uncertainty

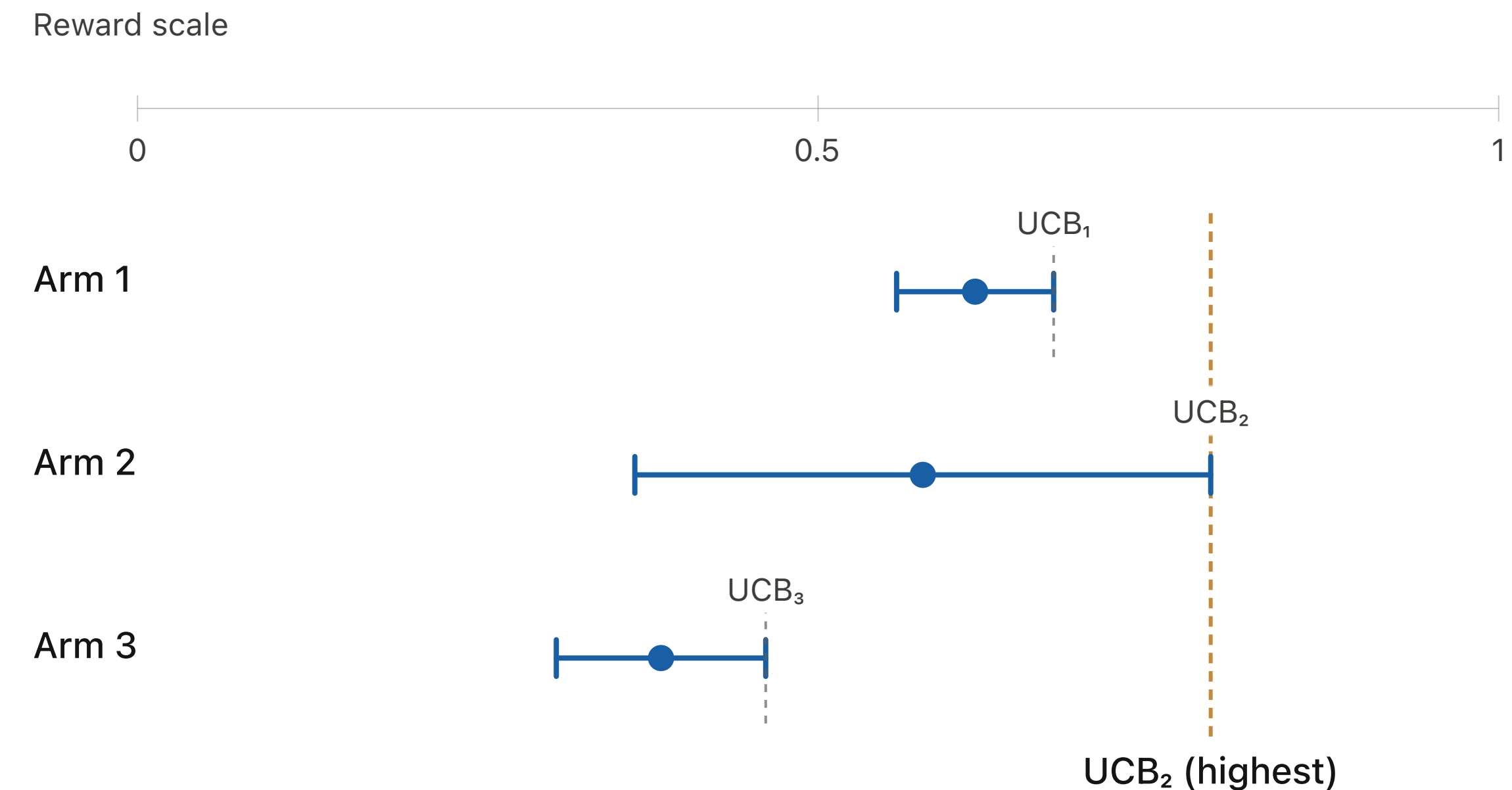
Act as if each arm is as good as it could reasonably be

UCB Algorithm

The decision rule

$$UCB_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{c \log T}{T_i(t)}}$$

$$\text{Pull } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t - 1)$$



Guiding Principle: Optimism in the Face of Uncertainty

Act as if each arm is as good as it could reasonably be

$\sqrt{c \log T / T_i(t)}$ is an exploration bonus term, added to mean estimate

Unlike Sequential Elimination, UCB does not wait until intervals separate

UCB Algorithm

Pseudocode

Initialise: Pull each arm once

For $t = K + 1, \dots, T$

For each arm i :

Compute $UCB_i(t - 1)$

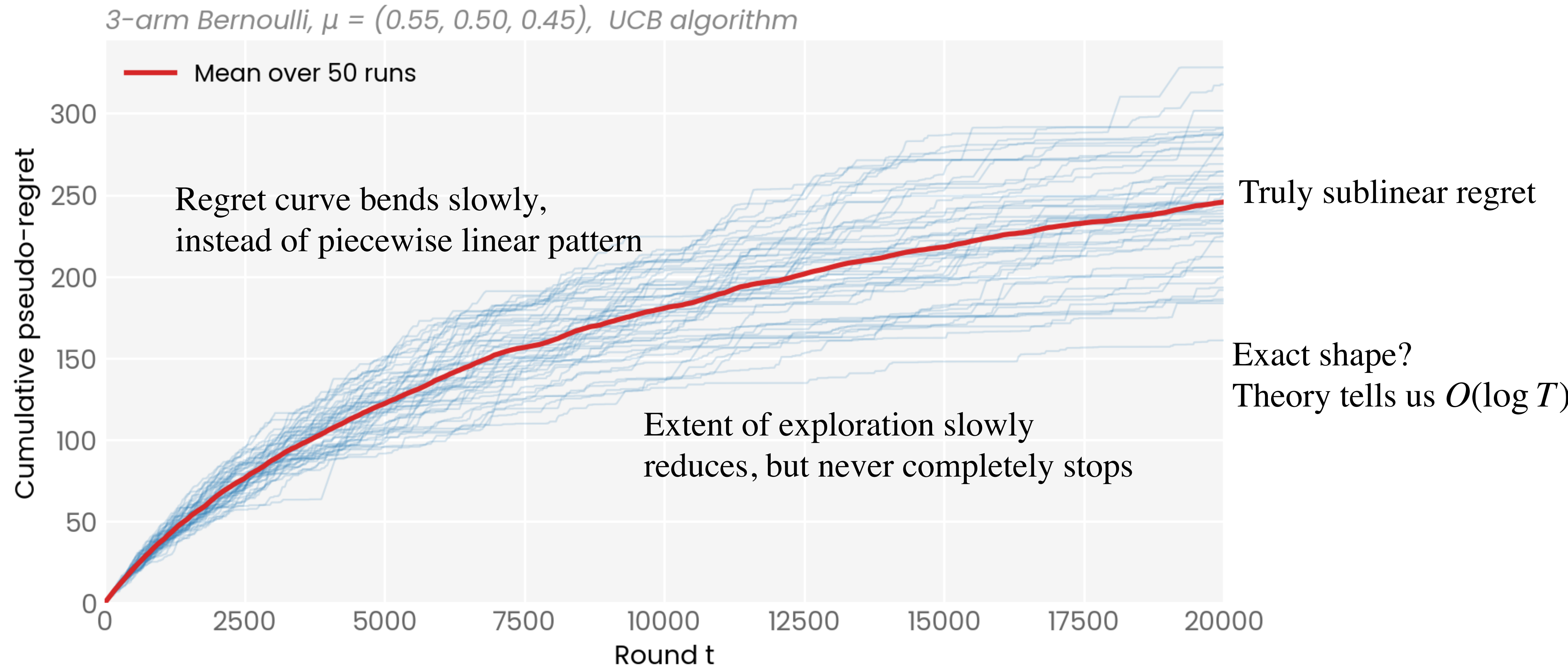
Pull arm $A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t - 1)$

Observe reward X_t

Update empirical mean and pull count of A_t

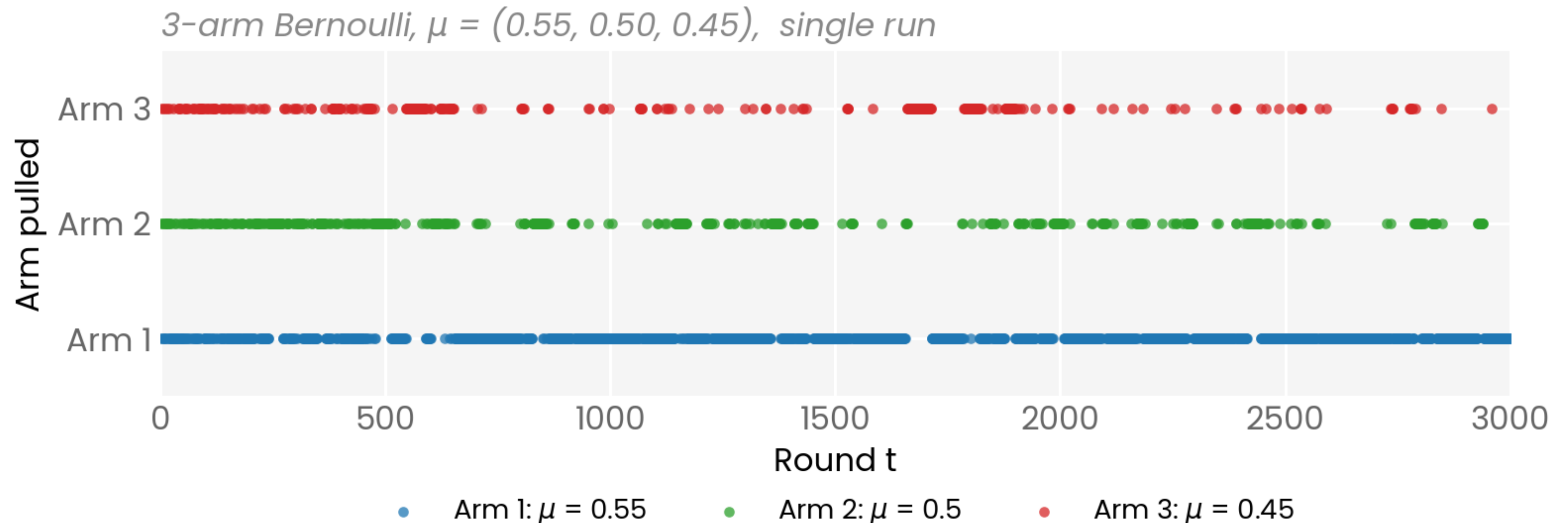
Empirical Performance

Cumulative regret curves



Empirical Performance

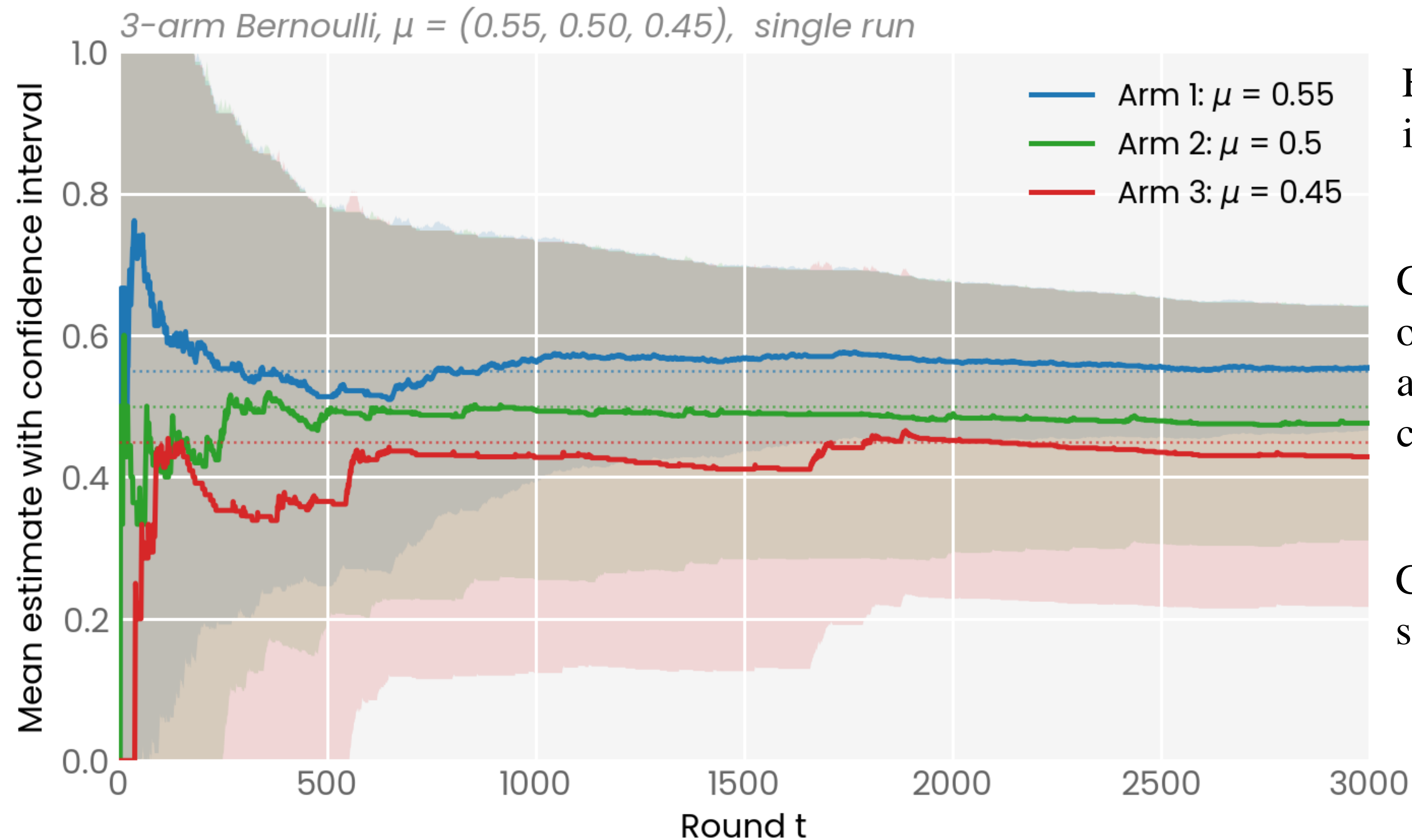
Arm Pulls Over Time



Pulls increasingly concentrate on the best arm. Important for sublinear regret!

Empirical Performance

Evolution of Confidence Intervals



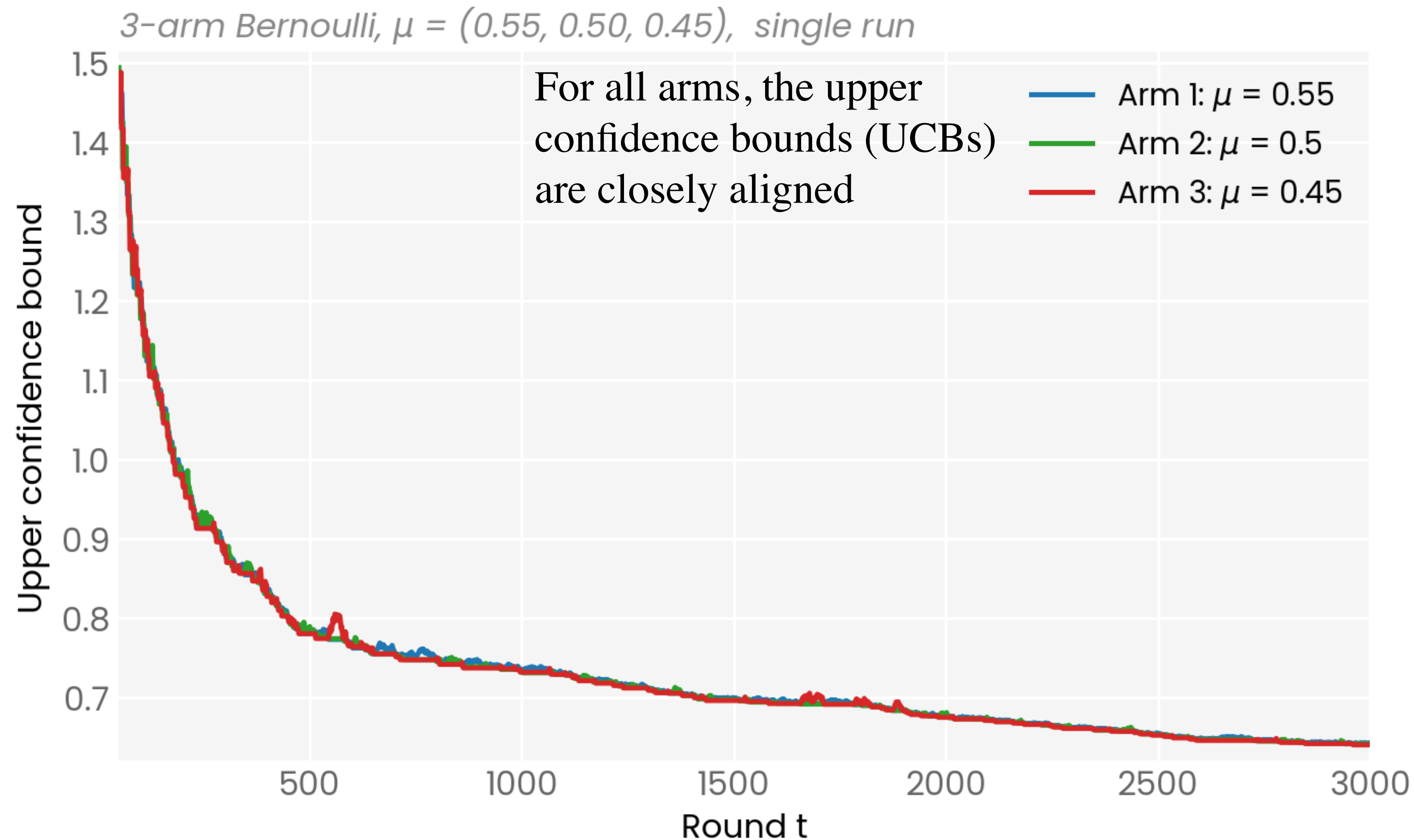
For all arms, confidence intervals shrink over time

Confidence interval of optimal arm is smallest, and empirical average has converged to true mean

Confidence intervals of suboptimal arms remain large

Empirical Performance

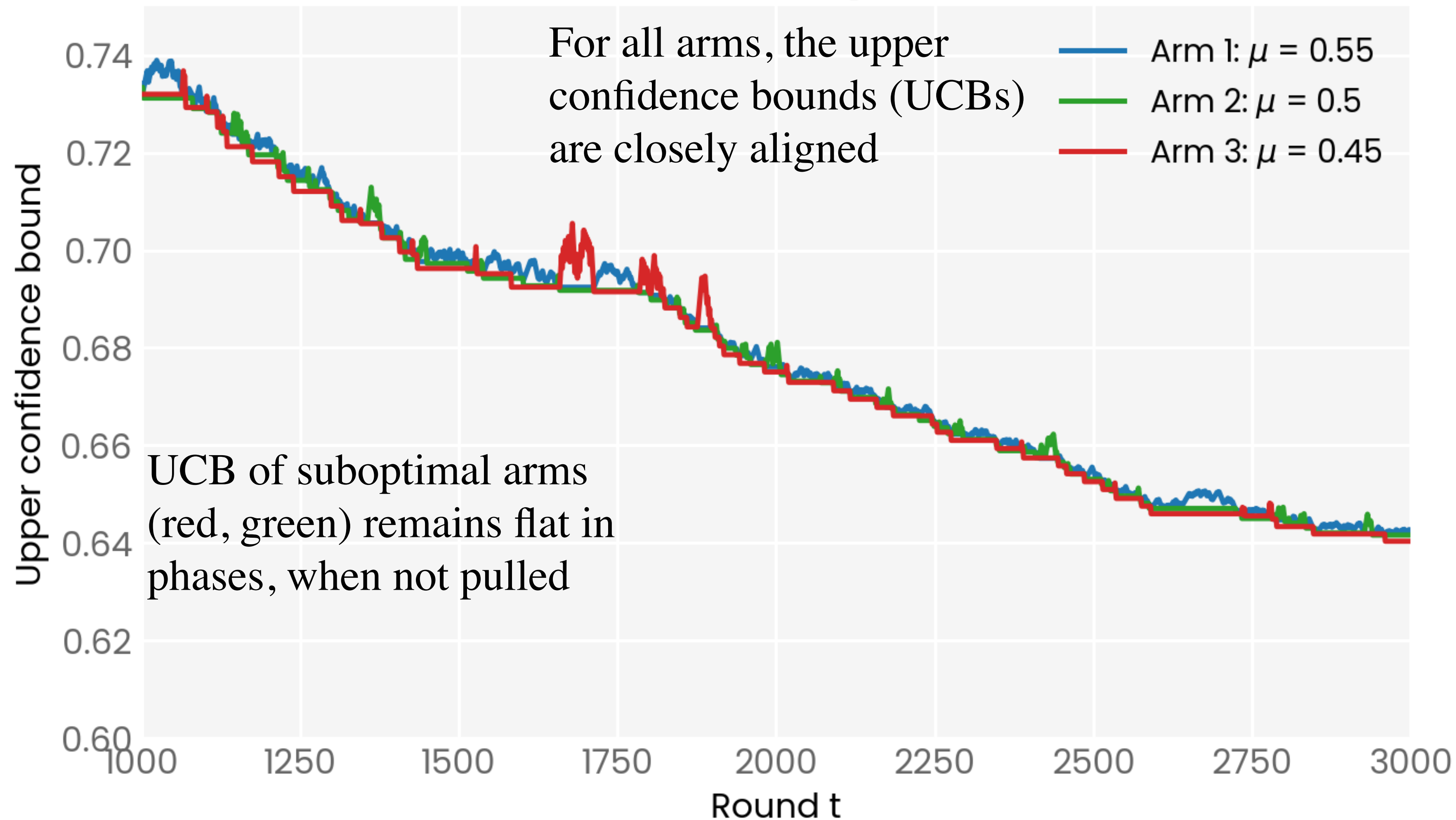
Tracking the UCB



Empirical Performance

Tracking the UCB

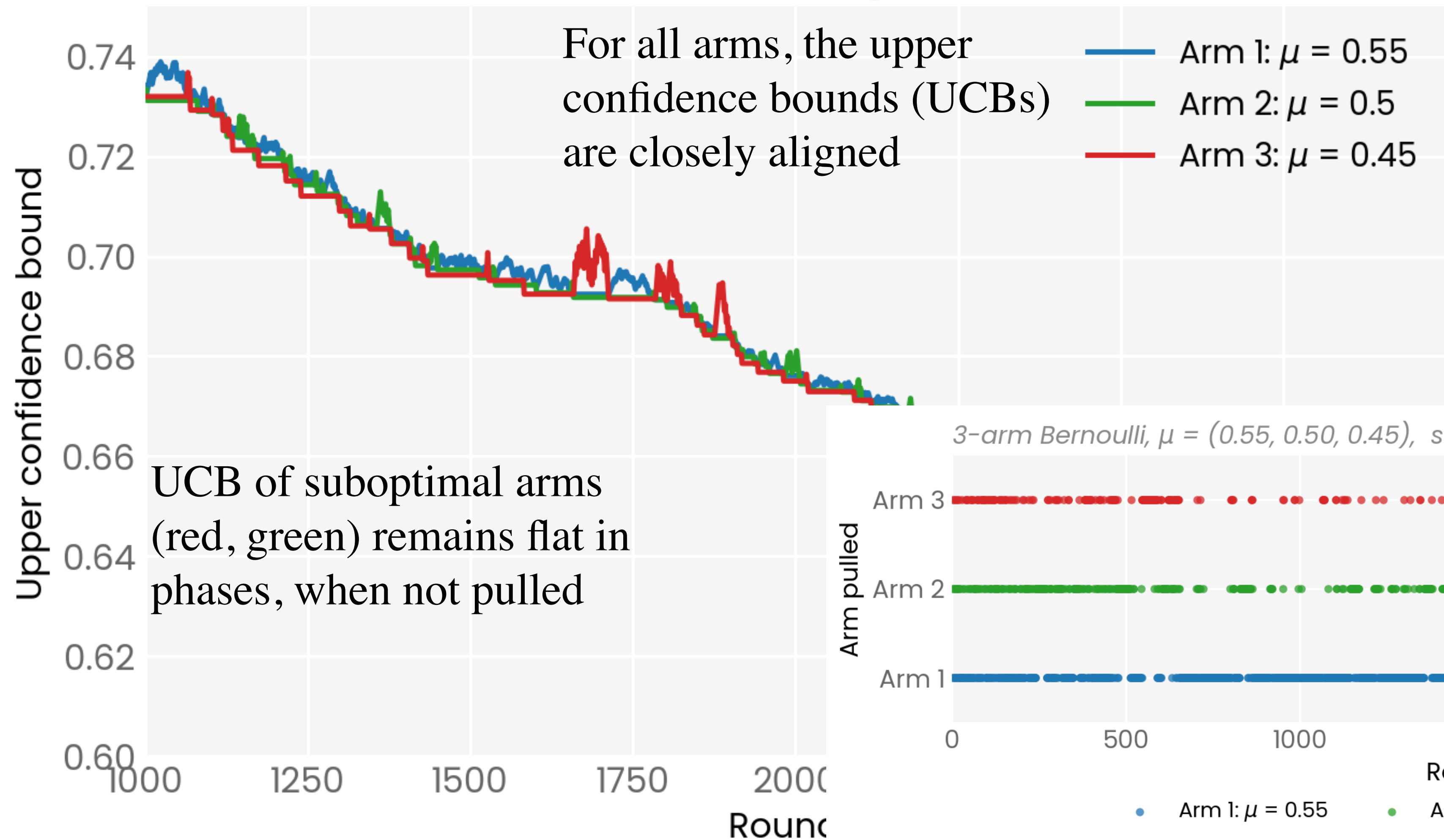
3-arm Bernoulli, $\mu = (0.55, 0.50, 0.45)$, single run, zoomed in



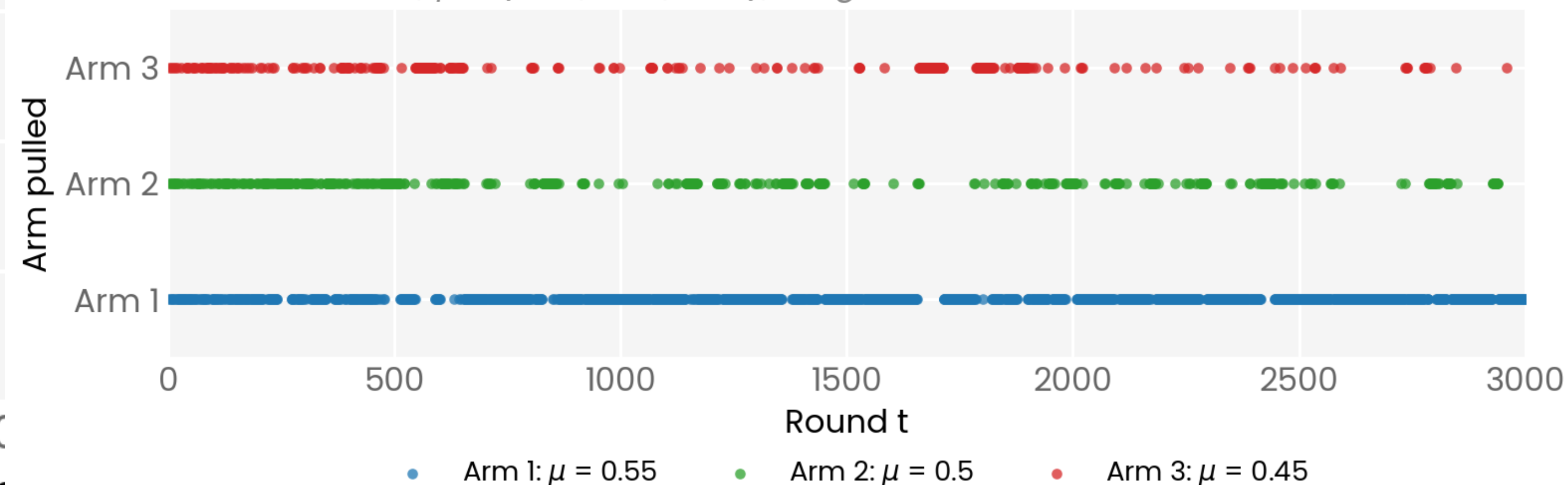
Empirical Performance

Tracking the UCB

3-arm Bernoulli, $\mu = (0.55, 0.50, 0.45)$, single run, zoomed in



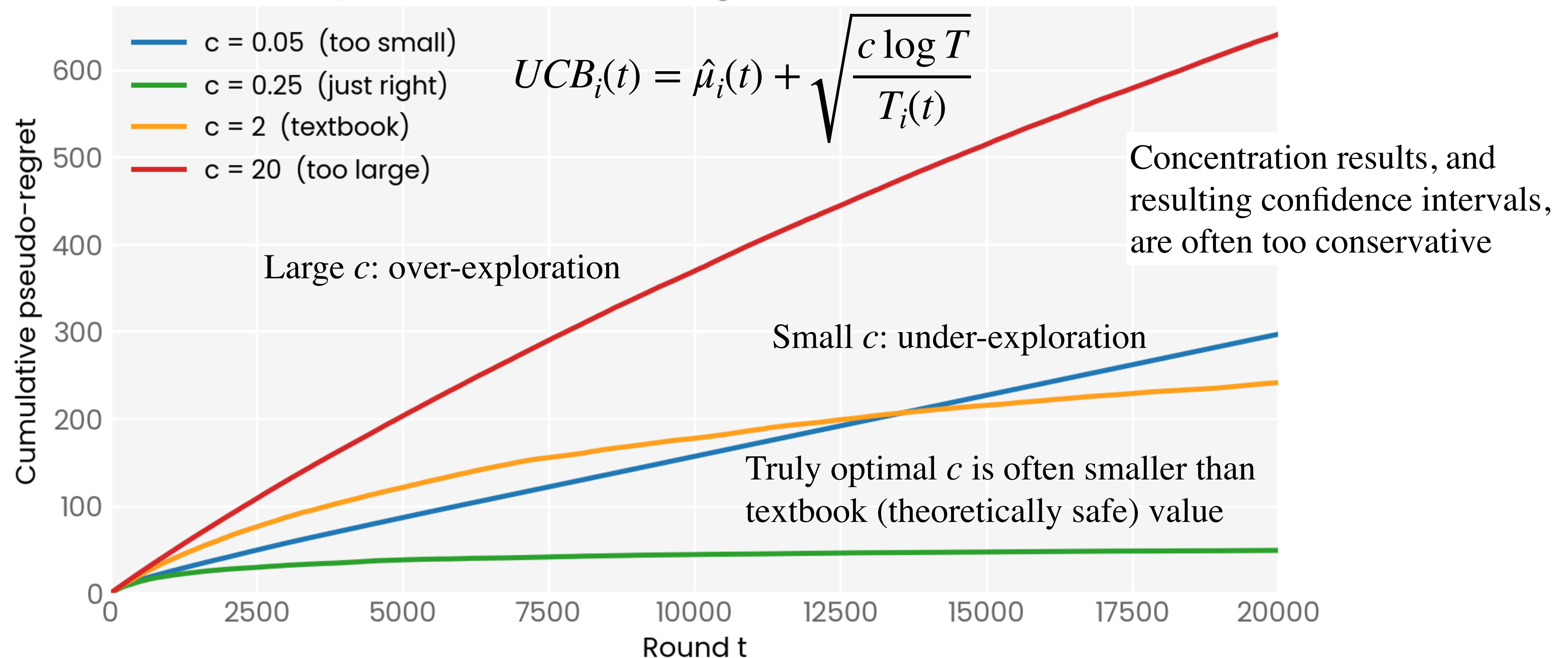
3-arm Bernoulli, $\mu = (0.55, 0.50, 0.45)$, single run



Empirical Performance

Exploration Hyperparameter

3-arm Bernoulli, $\mu = (0.55, 0.50, 0.45)$, averaged over 100 runs



Performance Guarantees

Intuition

$$UCB_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{c \log T}{T_i(t)}}$$

An arm can be selected if it has **high mean** or **high uncertainty**

Performance Guarantees

Intuition

$$UCB_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{c \log T}{T_i(t)}}$$

An arm can be selected if it has **high mean** or **high uncertainty**

Why does it minimise regret?

- With high probability, $UCB_{i^*}(t) \geq \mu^*$ always (concentration result)
- Arm $i \neq i^*$ is pulled $\Rightarrow UCB_i(t) \geq UCB_{i^*}(t)$
- With enough pulls, $UCB_i(t) \approx \mu_i$ ($< \mu^* \leq UCB_{i^*}(t)$)
- Suboptimal arms pulled not too many times!

Performance Guarantees

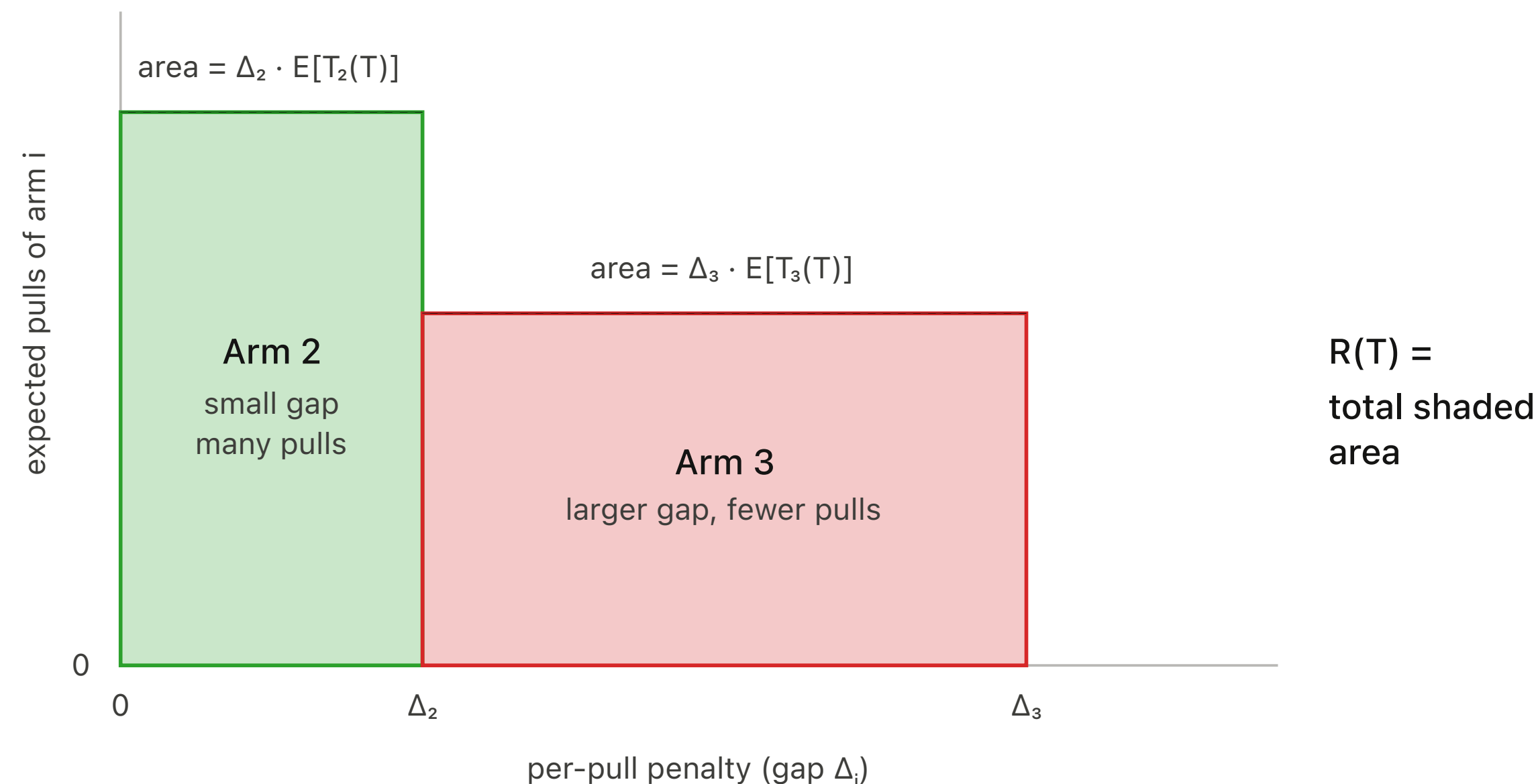
Starting from regret decomposition

- Recall from Lecture 3: $R(T) = \sum_{i=1}^K \Delta_i \mathbb{E}[T_i(T)]$

Performance Guarantees

Starting from regret decomposition

- Recall from Lecture 3: $R(T) = \sum_{i=1}^K \Delta_i \mathbb{E}[T_i(T)]$
- Performance guarantee: an upper bound on the regret
 - We need to find an upper bound on the number of times an arm is pulled



Performance Guarantees

Defining a good event

Hoeffding's inequality says:

with high probability, true mean of every arm sits inside its C.I. at every step

$$UCB_i(t) \geq \mu_i(t) \quad \forall i, \forall t \quad \text{with probability} \geq 1 - \delta$$

Performance Guarantees

Defining a good event

Hoeffding's inequality says:

with high probability, true mean of every arm sits inside its C.I. at every step

$$UCB_i(t) \geq \mu_i(t) \quad \forall i, \forall t \quad \text{with probability} \geq 1 - \delta$$

- In case of bad event, (if any of these inequalities are ever violated)
regret is at most T (all $|\mu_i - \mu_j| \leq 1$ for Bernoulli bandits)
- Bad event occurs with probability $\leq \delta = 1/T$

Performance Guarantees

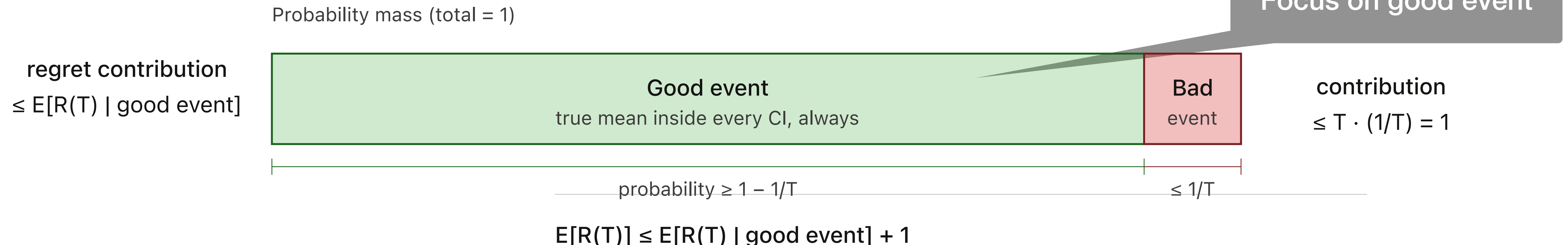
Defining a good event

Hoeffding's inequality says:

with high probability, true mean of every arm sits inside its C.I. at every step

$$UCB_i(t) \geq \mu_i(t) \quad \forall i, \forall t \quad \text{with probability} \geq 1 - \delta$$

- In case of bad event, (if any of these inequalities are ever violated)
regret is at most T (all $|\mu_i - \mu_j| \leq 1$ for Bernoulli bandits)
- Bad event occurs with probability $\leq \delta = 1/T$

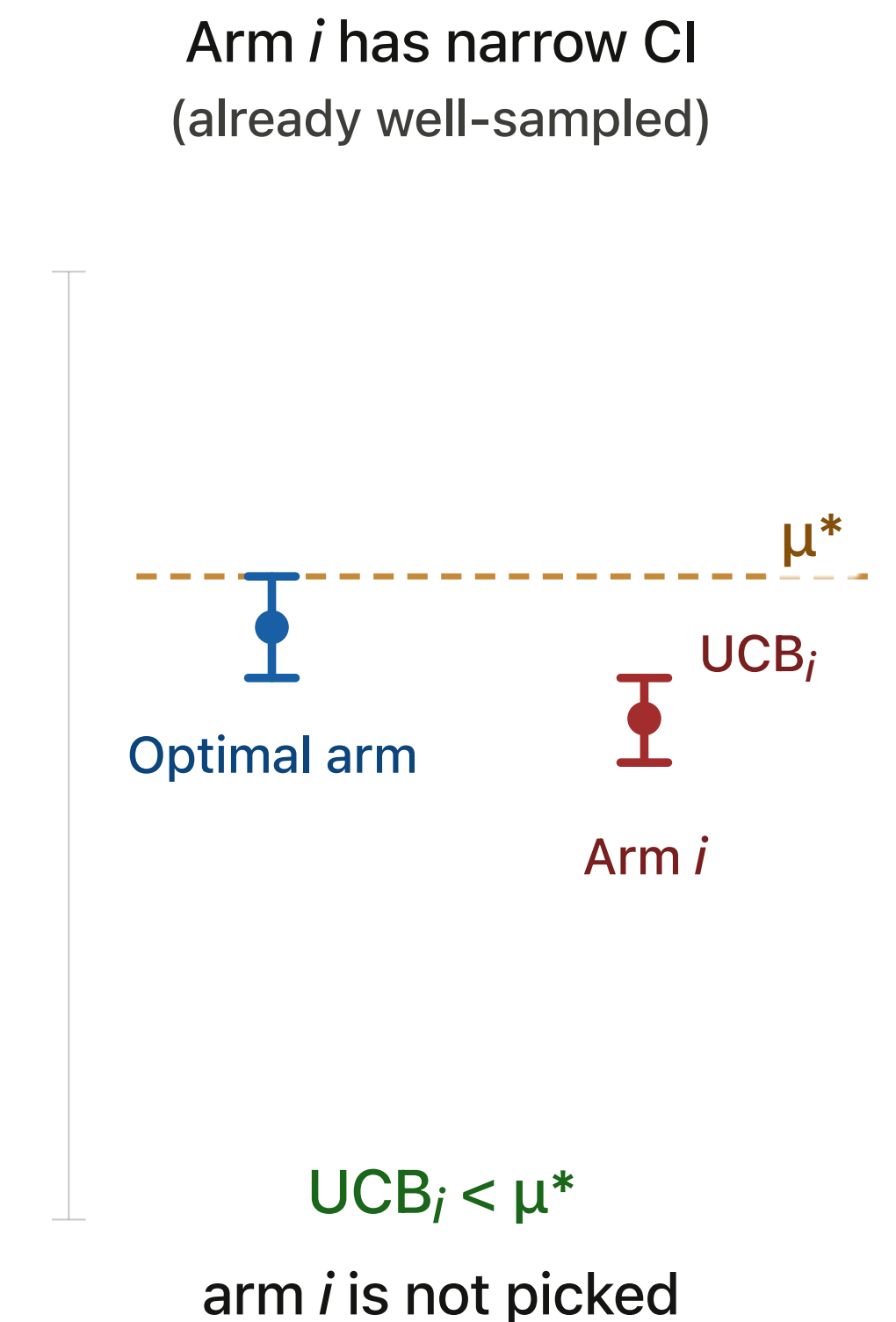
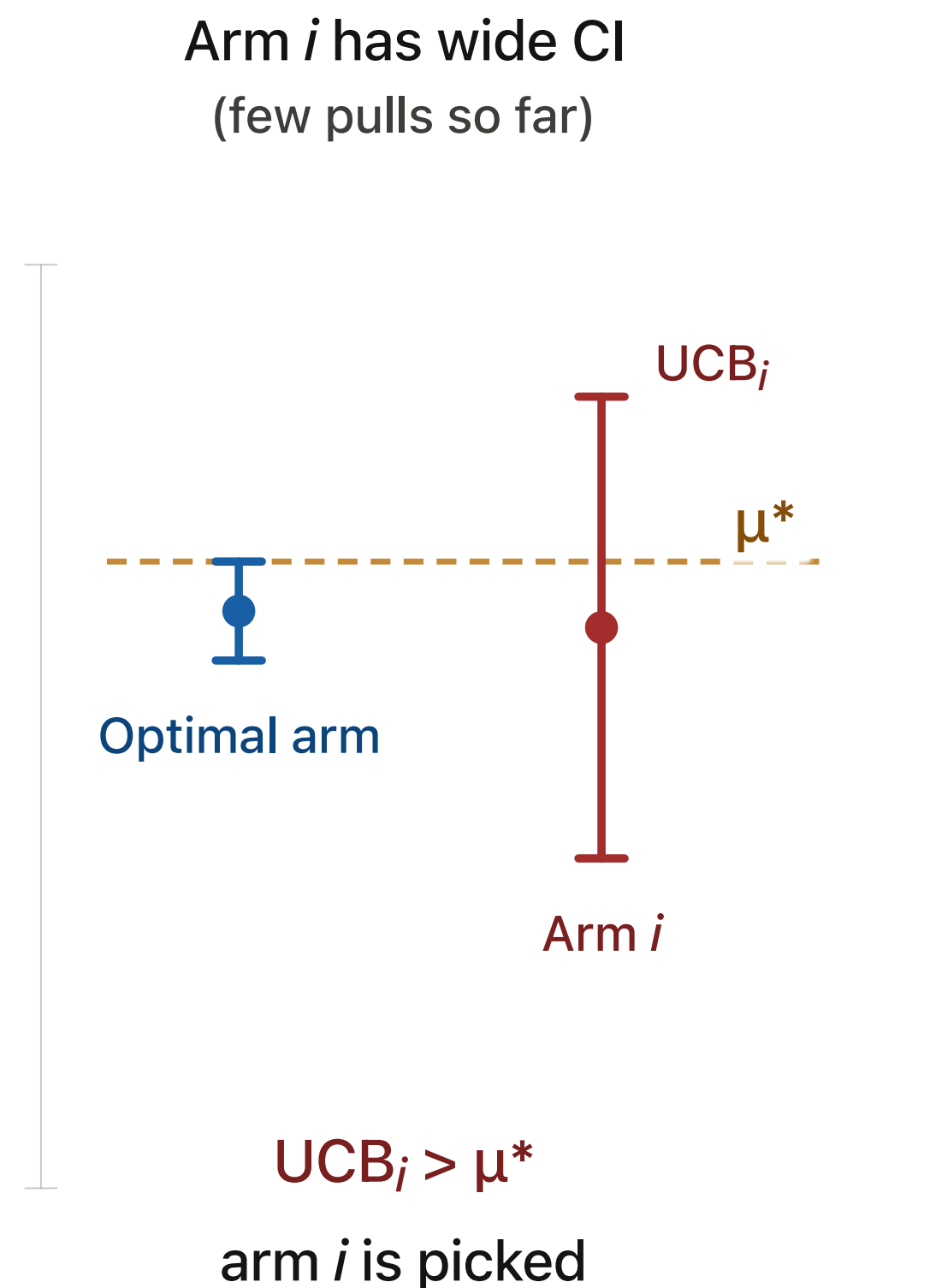


Performance Guarantees

When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} \text{UCB}_i(t-1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):



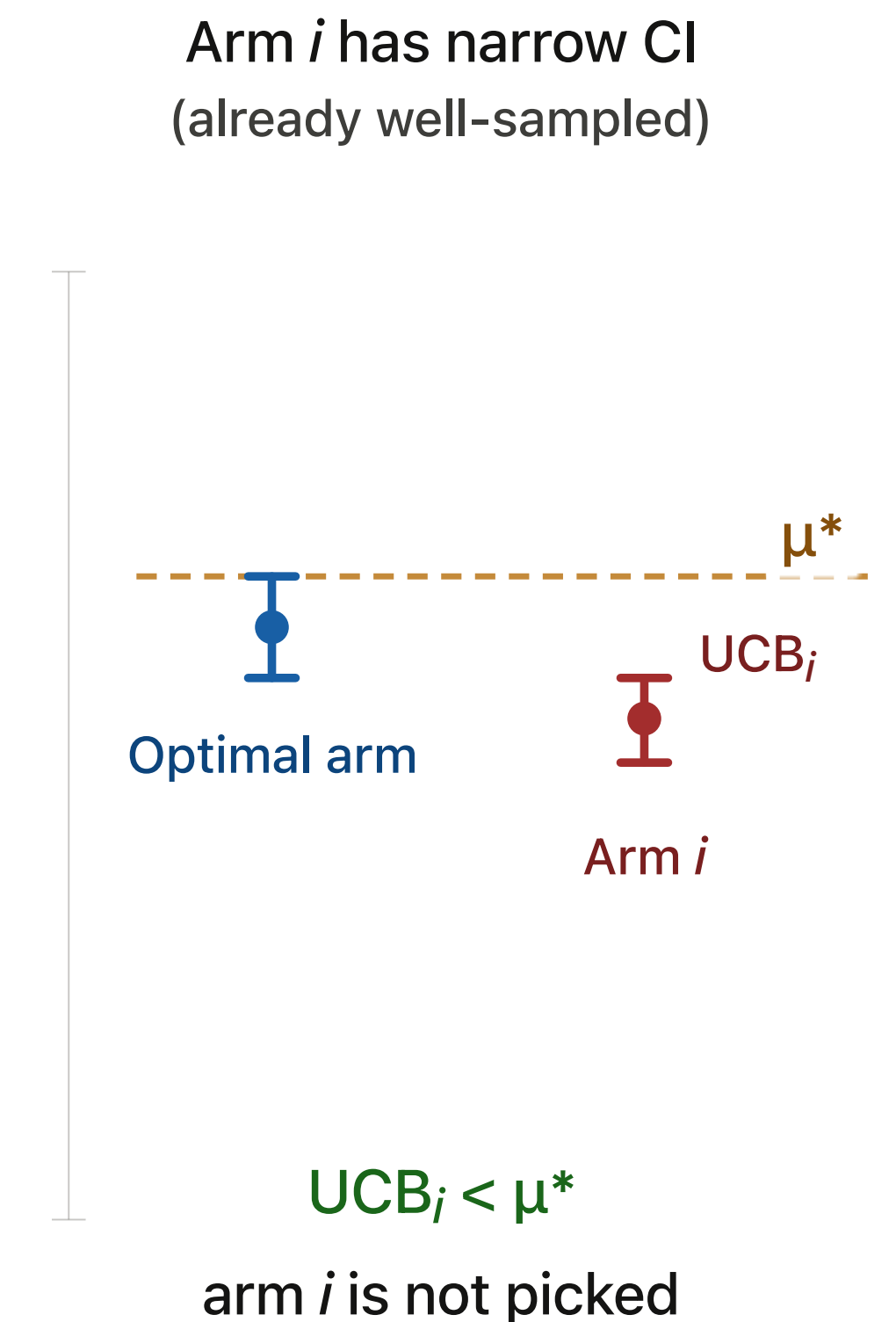
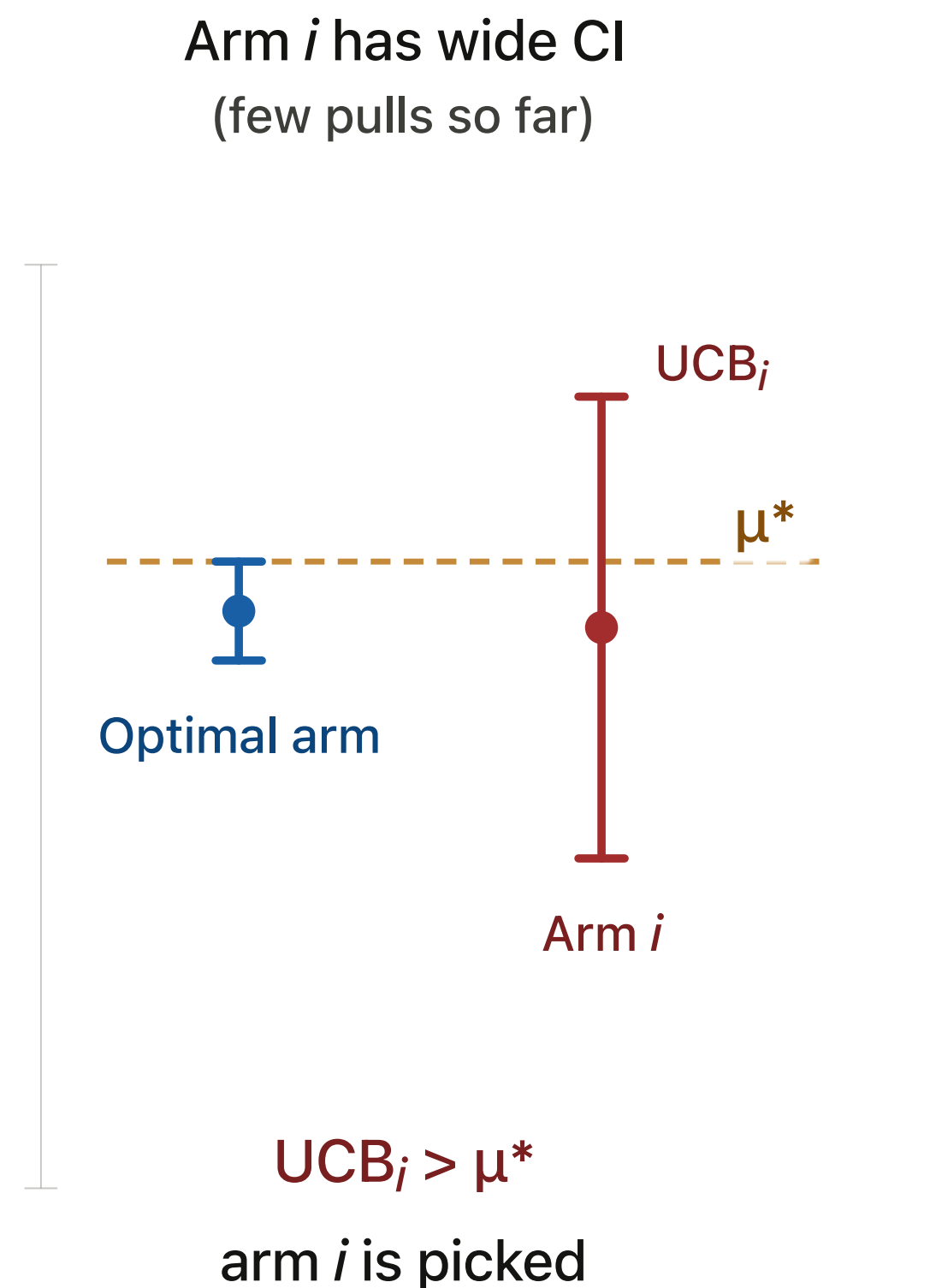
Performance Guarantees

When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} \text{UCB}_i(t - 1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):

- $\text{UCB}_i(t) \geq \text{UCB}_{i^*}(t)$



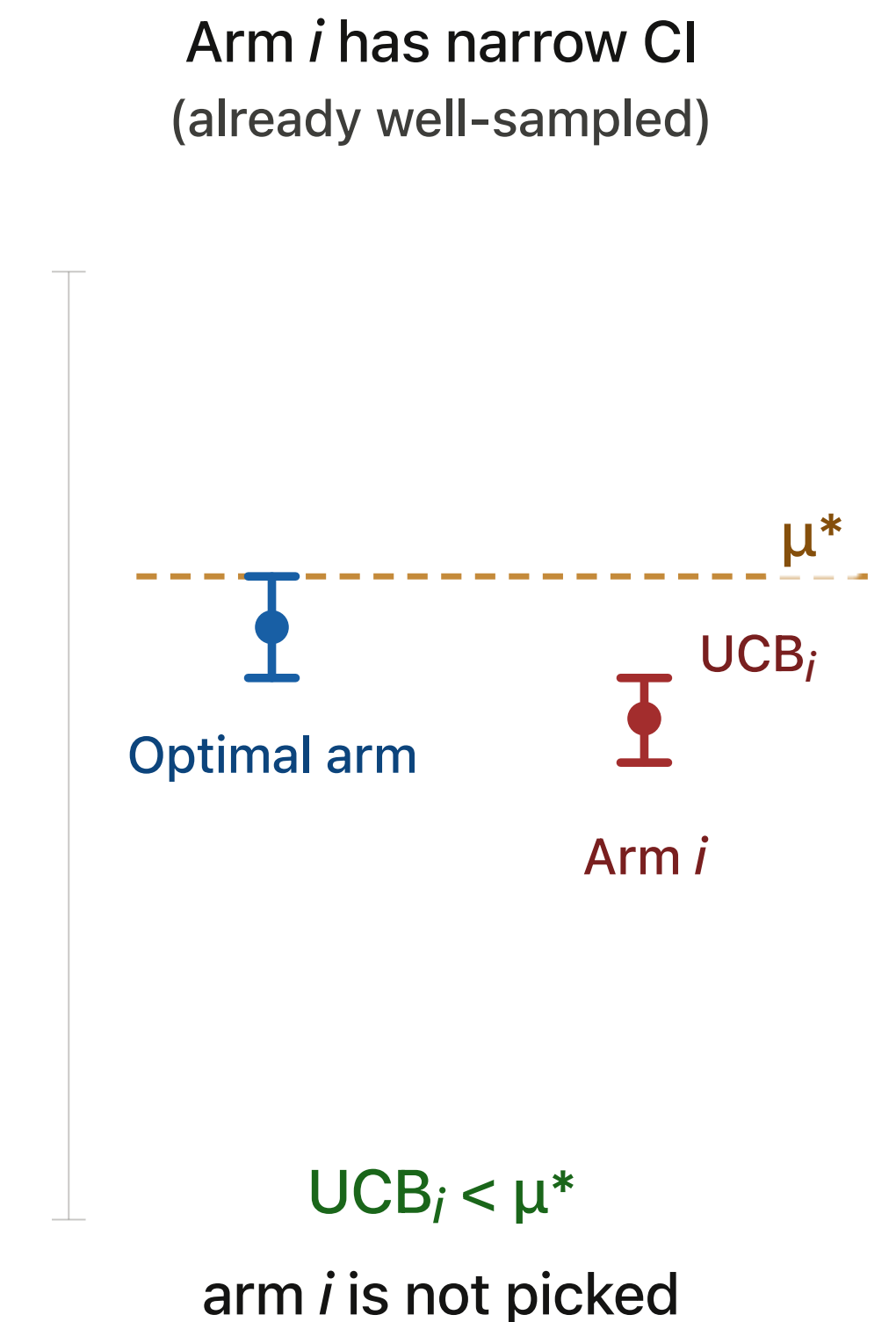
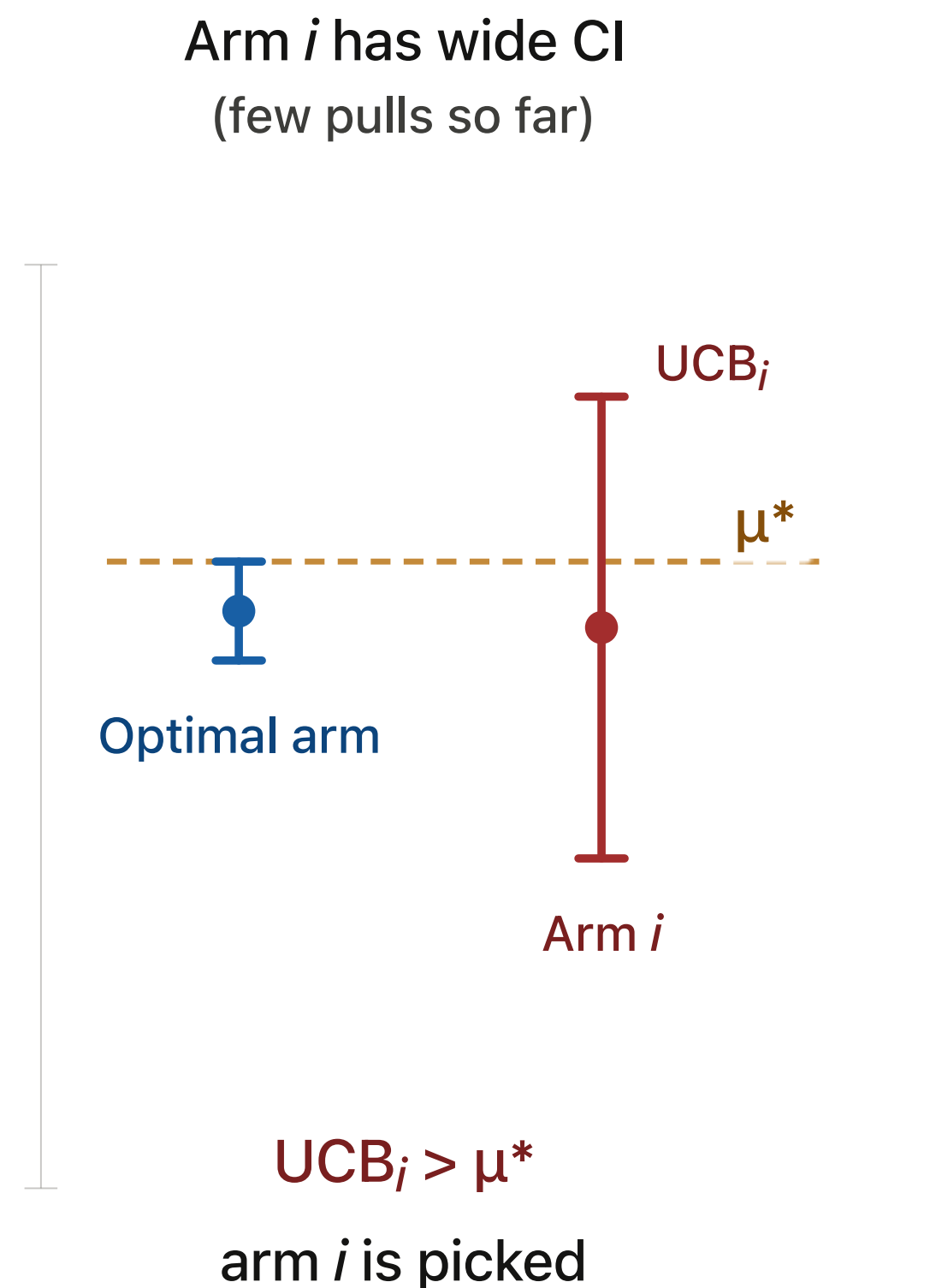
Performance Guarantees

When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} \text{UCB}_i(t - 1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):

- $\text{UCB}_i(t) \geq \text{UCB}_{i^*}(t)$
- $\text{UCB}_{i^*}(t) \geq \mu^*$



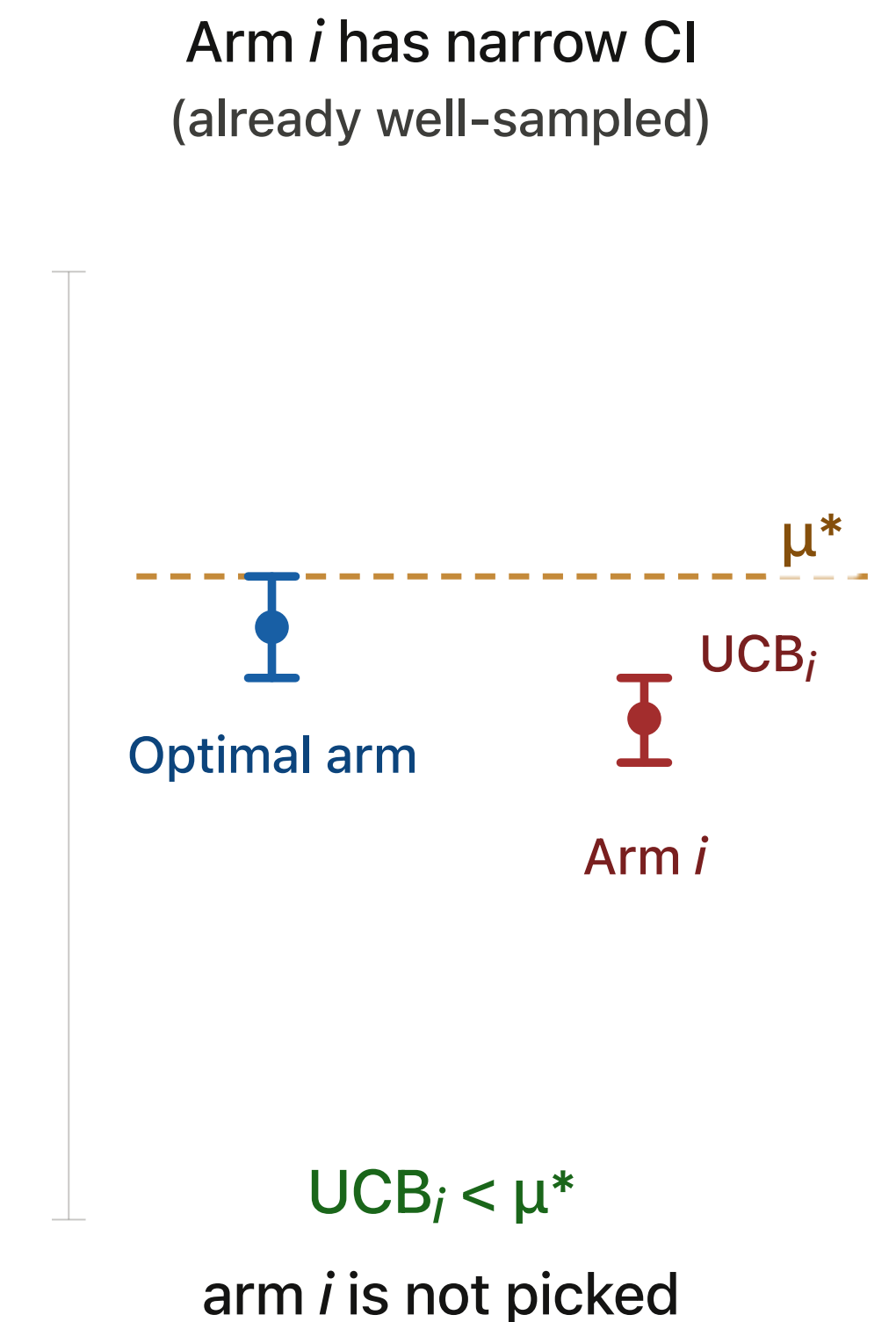
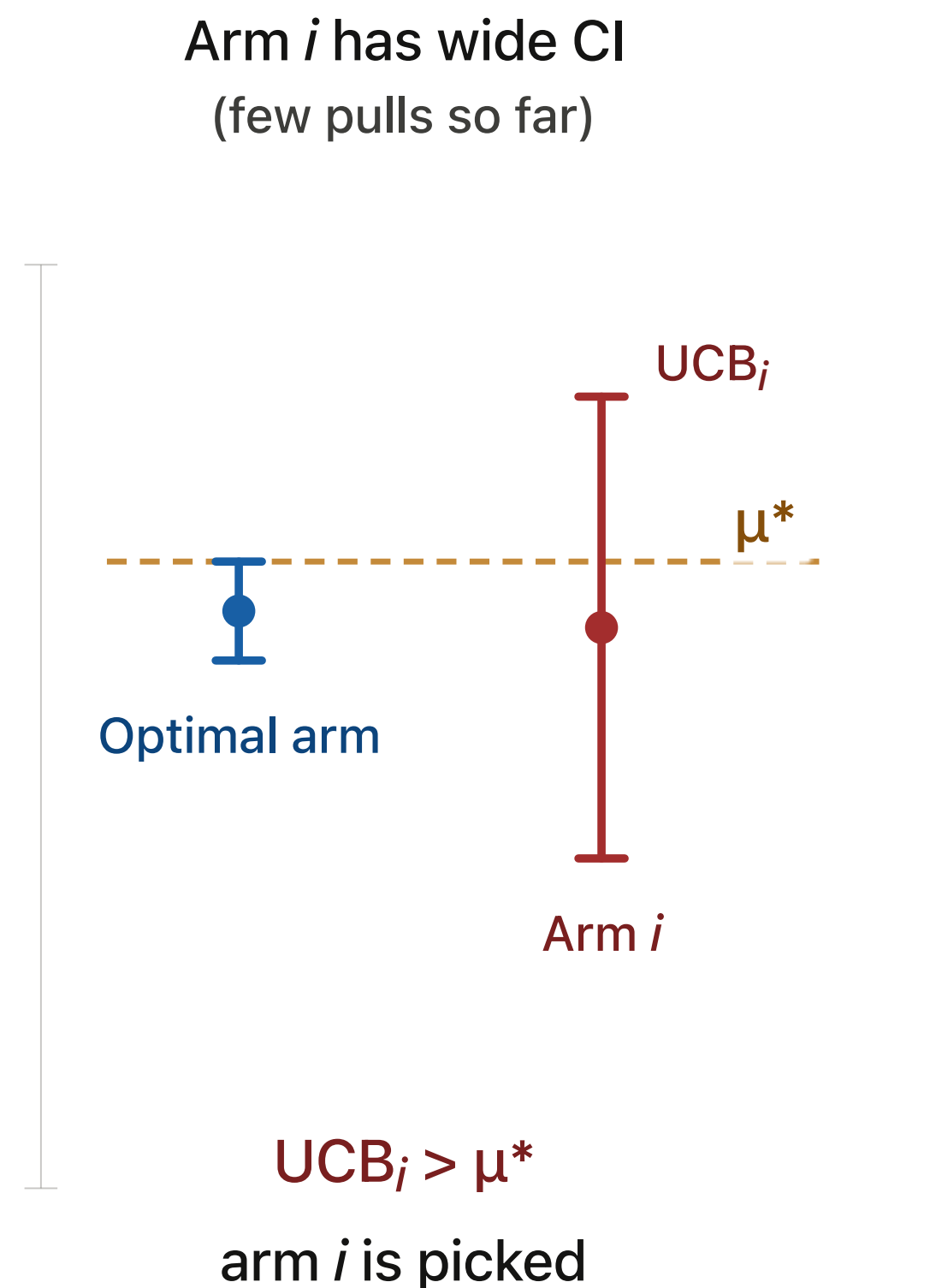
Performance Guarantees

When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t-1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):

- $UCB_i(t) \geq UCB_{i^*}(t)$
- $UCB_{i^*}(t) \geq \mu^*$
- $\Rightarrow UCB_i(t) \geq \mu^*$



Performance Guarantees

When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t - 1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):

- $UCB_i(t) \geq \mu^*$
- $UCB_i(t) = \hat{\mu}_i(t) + \text{bonus}_i(t)$

Performance Guarantees

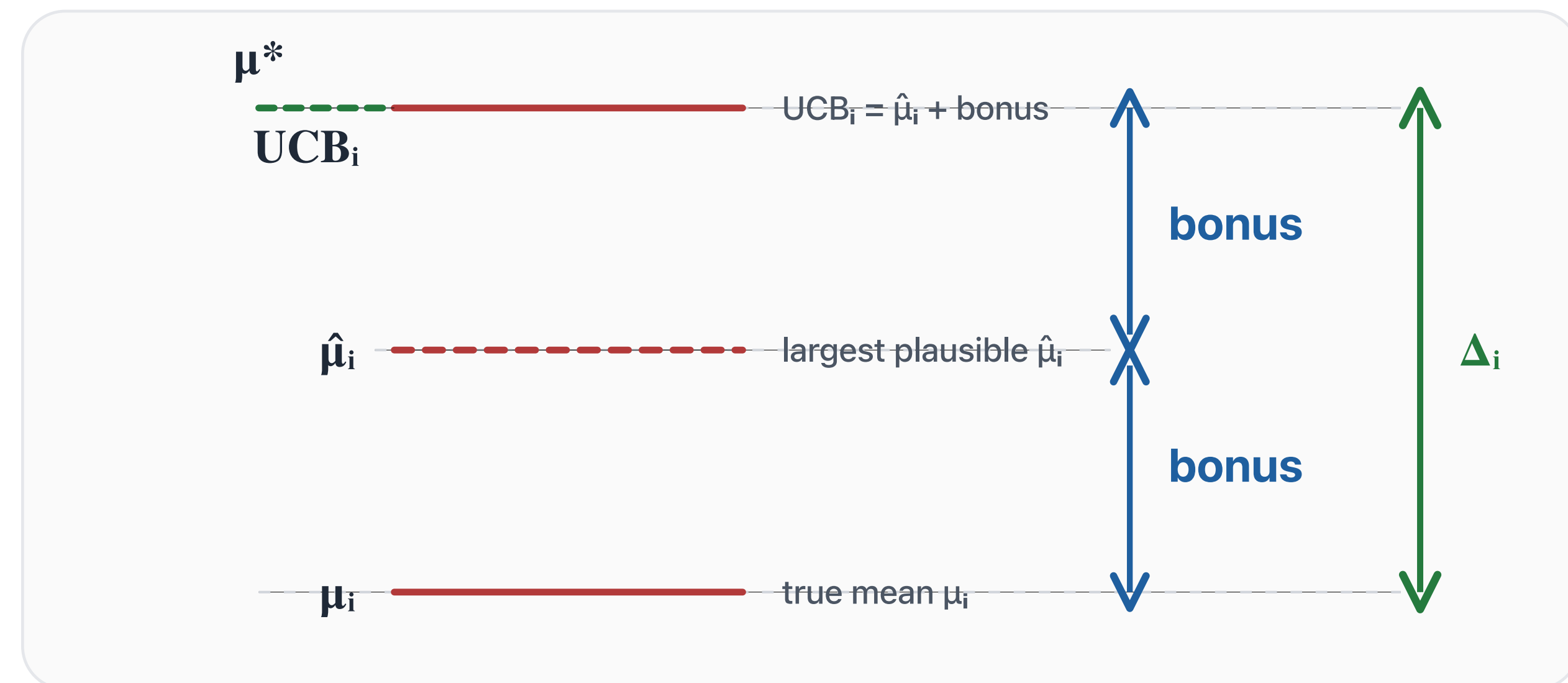
When can a bad arm be selected?

$$\text{UCB rule: } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t-1)$$

If arm i is suboptimal AND it is picked at time t (AND **good event** holds):

- $UCB_i(t) \geq \mu^*$
- $UCB_i(t) = \hat{\mu}_i(t) + \text{bonus}_i(t)$
- $\hat{\mu}_i(t) \leq \mu_i + \text{bonus}_i(t)$
- $\Rightarrow \text{bonus}_i(t) \geq \Delta_i / 2$

If suboptimal arm i is chosen, its UCB must be high enough to reach μ^*



$$2 \times \text{bonus} \geq \Delta_i \Leftrightarrow \text{bonus} \geq \Delta_i / 2$$

Performance Guarantees

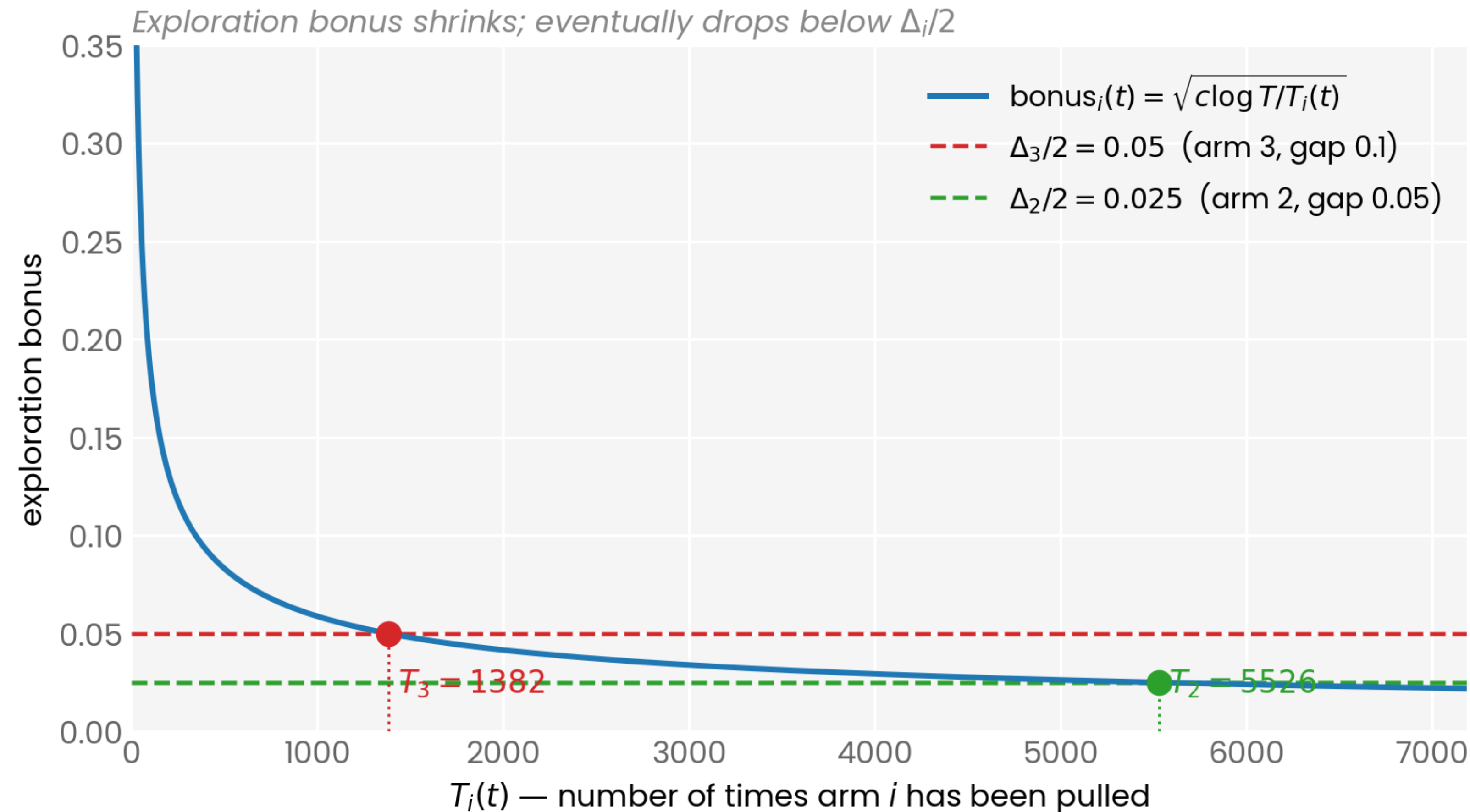
Bounding suboptimal arm pulls

- $\text{bonus}_i(t) \geq \Delta_i/2$
- $\text{bonus}_i(t) = \sqrt{\frac{c \log T}{T_i(T)}}$
- $\sqrt{\frac{c \log T}{T_i(T)}} \geq \frac{\Delta_i}{2}$

Performance Guarantees

Bounding suboptimal arm pulls

- $\text{bonus}_i(t) \geq \Delta_i/2$
- $\text{bonus}_i(t) = \sqrt{\frac{c \log T}{T_i(T)}}$
- $\sqrt{\frac{c \log T}{T_i(T)}} \geq \frac{\Delta_i}{2}$
- $\Rightarrow T_i(T) \leq 4c \frac{\log T}{\Delta_i^2}$



Performance Guarantee

Final Regret Bound

$$R(T) = \sum_{i \in \{1, 2, \dots, K\}} \Delta_i \mathbb{E}[T_i(T)] \leq \sum_{i \in \{1, 2, \dots, K\}} 4c \frac{\log T}{\Delta_i}$$

Performance Guarantee

Final Regret Bound

$$R(T) = \sum_{i \in \{1, 2, \dots, K\}} \Delta_i \mathbb{E}[T_i(T)] \leq \sum_{i \in \{1, 2, \dots, K\}} 4c \frac{\log T}{\Delta_i}$$

- Gets $O(\log T / \Delta)$ regret, without knowing Δ
 - *ETC gets this regret only if we know Δ*
- Per-arm cost is $O(1/\Delta_i)$: gets pulled $O(1/\Delta_i^2)$ times, but contributes less (Δ_i) per wrong pull
- UCB adapts to the right level of exploration, because decisions are made based on confidence intervals

Summary

What We Learned Today

- ETC and ε -greedy both had linearly increasing regret
 - Because of *a priori* rules for exploration vs exploitation
- UCB algorithm: used data to smoothly decide between exploration and exploitation

$$UCB_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{c \log T}{T_i(t)}}$$

$$\text{Pull } A_t = \arg \max_{i \in \{1, 2, \dots, K\}} UCB_i(t - 1)$$

Summary

What We Learned Today

- We analysed the performance of UCB
 - Proved that number of bad arm pulls $\sim \frac{\log T}{\Delta^2}$
 - Logic based purely on confidence intervals, coming from Hoeffding's inequality
 - Implies regret $\sim \frac{\log T}{\Delta}$
- UCB uses the optimism in the face of uncertainty principle
- Next time: Thompson Sampling (another algorithm with the same principle)